UNIVERSIDAD SAN FRANCISCO DE QUITO USFQ

Organización de Computadores



Implementación de Buscaminas en Arquitectura MARIE

Joshua Reinoso, Karina Dahik, Gabriel Cazares

Implementación de Buscaminas en Arquitectura MARIE

Joshua Reinoso, Karina Dahik, Gabriel Cazares

***Resumen*—Este trabajo presenta el desarrollo de una versión funcional del juego Buscaminas implementado en la arquitectura MARIE (*Machine Architecture that is Really Intuitive and Easy*). El proyecto integra un generador de mapas en Python con una implementación en lenguaje ensamblador MARIE, permitiendo representar un tablero dinámico de 16×16 celdas, gestionar la interacción del usuario y aplicar lógica de juego completa: expansión automática tipo DFS, detección de minas, sistema de banderas y verificación de victoria. Se analiza el uso de estructuras de memoria segmentadas, la simulación de operaciones aritméticas ausentes en MARIE (multiplicación mediante sumas repetidas), y la aplicación de patrones de diseño en un entorno de bajo nivel. Los resultados evidencian la viabilidad de implementar lógica compleja en arquitecturas simples mediante una adecuada organización de datos y control de flujo.**

***Index Terms*—MARIE, ensamblador, buscaminas, arquitectura de computadores, diseño de software, simulación, DFS, RGB565**

I. Introducción

El juego Buscaminas es un problema clásico que combina lógica, manejo de estructuras de datos y control de flujo. En este proyecto se implementa una versión completa del juego utilizando la arquitectura MARIE, un modelo educativo de 16 bits diseñado para comprender el funcionamiento interno de un computador [2].

A diferencia de implementaciones modernas, MARIE presenta limitaciones importantes: ausencia de multiplicación nativa, sin soporte para estructuras dinámicas, sin pila de llamadas, y un conjunto de instrucciones reducido a 13 operaciones básicas [3]. Por esta razón, el proyecto combina dos componentes complementarios:

- Un generador de mapas en Python (`python_map_gen.py`) que construye y precomputa todos los datos del tablero.
- Un programa en ensamblador MARIE (`BombGame.mas` / `juego.mas`) que ejecuta la lógica completa del juego.

El objetivo principal es demostrar cómo un sistema con recursos limitados puede ejecutar lógica compleja mediante una adecuada organización de memoria, precomputación de datos y algoritmos adaptados al conjunto de instrucciones disponible.

II. Arquitectura del Sistema

El sistema sigue una arquitectura de dos fases claramente separadas, como se ilustra en la Figura 1.

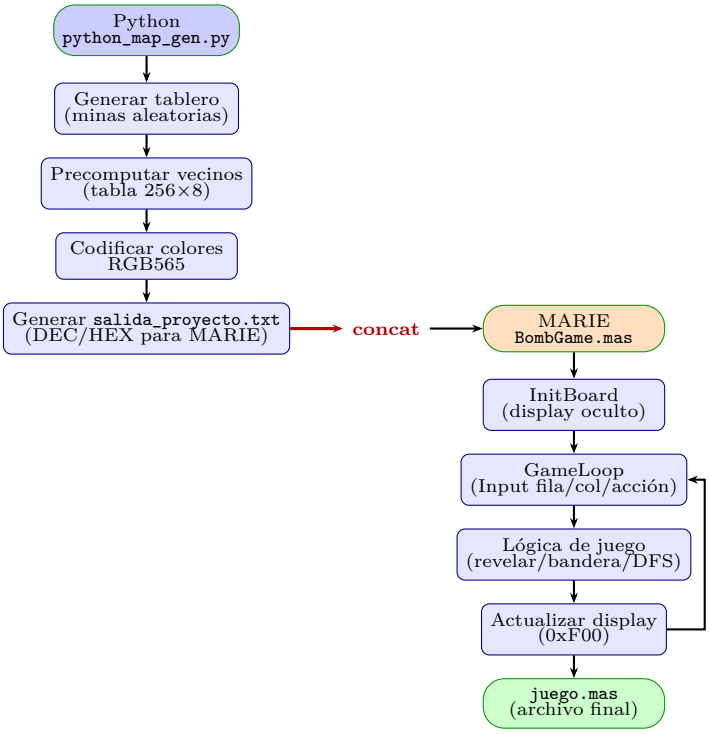


Figura 1. Arquitectura general del sistema: fase de generación (Python) y fase de ejecución (MARIE).

II-A. Generación del Tablero (Python)

El archivo `python_map_gen.py` implementa cinco funciones principales:

- **`build_board()`**: genera el tablero 16×16 con minas aleatorias y calcula los valores numéricos de cada celda (0–8, -1 para minas).
- **`get_neighbors(index)`**: precomputa la tabla de vecinos para cada celda. Cada entrada contiene hasta 8 índices lineales de celdas adyacentes; las posiciones inválidas se rellenan con -1.
- **`build_marie_block(board)`**: serializa el tablero en bloques de instrucciones `DEC`/`HEX` listos para MARIE.
- **`save_board_image(board, path)`**: genera la imagen `tablero_real.png` con colores RGB reales.
- **`build_game_file(base_dir, salida)`**: concatena `BombGame.mas` con el bloque de datos generado, produciendo `juego.mas`.

La salida principal es `salida_proyecto.txt`, que contiene los cinco segmentos de memoria descritos en la Sección III.

II-B. *Ejecución del Juego (MARIE)*

El archivo `BombGame.mas` contiene la lógica completa del juego organizada en subrutinas invocadas con `JNS`:

- `InitBoard`: inicializa el display con color oculto (`0x39E7`) y el mapa de visibilidad en cero.
- `GameLoop`: ciclo principal; lee fila, columna y acción del usuario.
- `MulRoutine`: multiplica mediante sumas repetidas (suple la ausencia de `MUL` en MARIE).
- `DoReveal` / `FloodStart` / `FloodLoop`: lógica de revelado y expansión DFS.
- `DoFlag` / `PutFlag` / `RemoveFlag`: gestión de banderas y puntuación.
- `Boom`: revela todas las minas al perder.
- `CheckVictory`: compara el puntaje con el total de minas.
- `PrintString` / `PrintSigned`: salida de texto y números con signo por consola.

III. Organización de Memoria

La clave del proyecto es la segmentación de memoria en bloques fijos de 256 palabras (una por celda del tablero 16×16). La Figura 2 muestra el mapa completo.

0x000	Código MARIE (lógica del juego)
0x120	Código + constantes + mensajes
0x238	Tablero real
0x338	Mapa de minas
0x438	Colores RGB565
0x538	Estado visible
0x638	Tabla de vecinos
0xE00	Stack DFS manual
0xF00	Display (memoria de video)

Figura 2. Mapa de memoria del sistema.

Los estados de visibilidad codificados en `0x538` son: **0** = oculta, **1** = revelada, **2** = en cola DFS, **3** = bandera.

La tabla de vecinos en `0x638` ocupa $256 \times 8 = 2048$ palabras. Para la celda i, su base se calcula como:

$$\text{NeighborBase} = i \times 8 + \texttt{0x638}$$

donde la multiplicación se realiza mediante `MulRoutine`.

IV. Flujo Principal del Juego

IV-A. *Ciclo de Juego y Cálculo de Índice*

El usuario ingresa tres valores: fila (r), columna (c) y acción (a: 1 = revelar, 2 = bandera). El índice lineal se calcula como:

$$\text{index} = r \times 16 + c$$

Dado que MARIE carece de instrucción de multiplicación, se implementa `MulRoutine`: suma `MulA` exactamente `MulB` veces usando un contador auxiliar.

Listing 1. Rutina de multiplicación por sumas repetidas.

```
MulRoutine, HEX 0
            LOAD Zero
            STORE MulResult
            STORE MulCount
MulL, LOAD MulCount
            SUBT MulB
            SKIPCOND 400 / salta si MulCount == MulB
            JUMP MulBody
            JUMPI MulRoutine
MulBody, LOAD MulResult
            ADD MulA
            STORE MulResult
            LOAD MulCount
            ADD One
            STORE MulCount
            JUMP MulL
```

IV-B. *Flujo de Decisión por Acción*

La Figura 3 muestra el flujo completo del ciclo de juego, incluyendo la bifurcación entre revelar y colocar bandera.

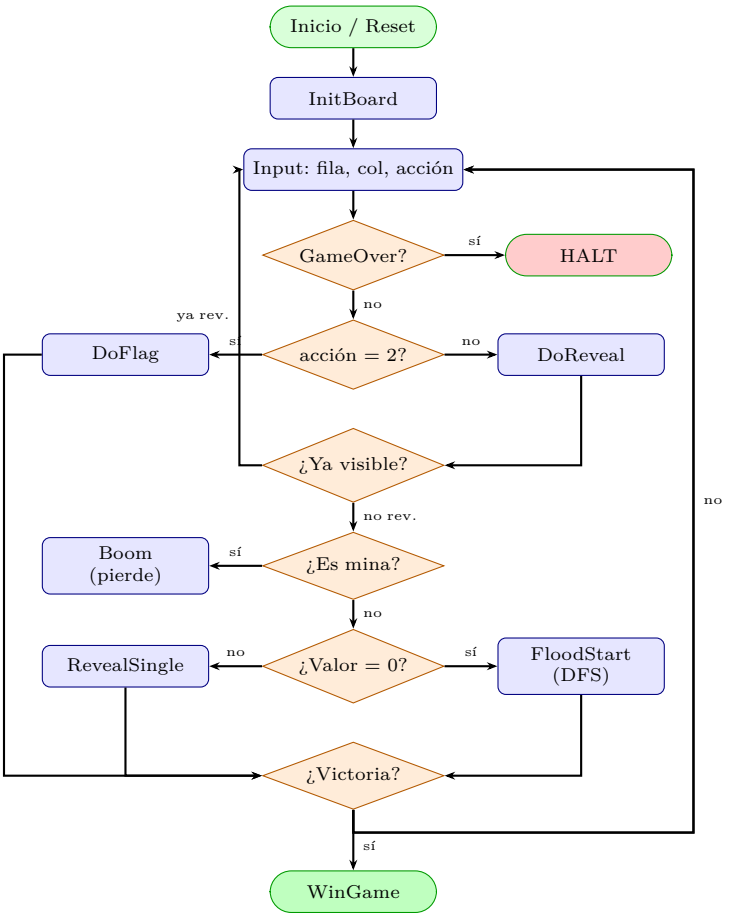


Figura 3. Flujo principal del ciclo de juego en MARIE.

IV-C. *Revelado de Celdas*

La subrutina `RevealCell` marca la celda como visible (`Estado[i] = 1`), lee el color precomputado desde `0x438` y lo escribe en el display (`0xF00`):

Listing 2. Subrutina RevealCell.

```
RevealCell, HEX 0
            LOAD CurrCell
            ADD AddrVisible
            STORE TmpAddr
            LOAD One
            STOREI TmpAddr / Estado = 1
            LOAD CurrCell
            ADD AddrColors
            STORE TmpAddr
            LOADI TmpAddr
            STORE TmpColor / leer color RGB565
            LOAD CurrCell
            ADD AddrDisplay
            STORE TmpAddr
            LOAD TmpColor
            STOREI TmpAddr / escribir en display
            JUMPI RevealCell
```

IV-D. *Expansión DFS (Flood Fill)*

Cuando se revela una celda con valor 0, se activa un algoritmo DFS iterativo usando un stack manual en `0xE00`. La Figura 4 detalla el flujo.

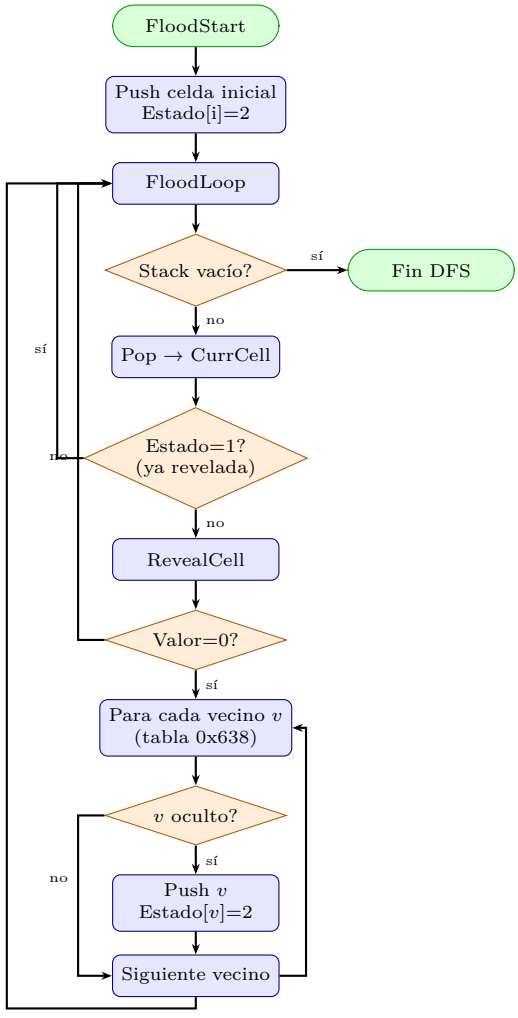


Figura 4. Algoritmo DFS iterativo para expansión de celdas vacías.

El stack se implementa como un arreglo en `0xE00` con un puntero `StackTop`. Las operaciones push/pop se realizan con `STOREI`/`LOADI` sobre la dirección calculada `AddrStack + StackTop`.

IV-E. *Sistema de Banderas y Puntuación*

El sistema de banderas implementa la mecánica de victoria del Buscaminas: el jugador debe marcar exactamente las minas. La Figura 5 muestra la lógica de puntuación.

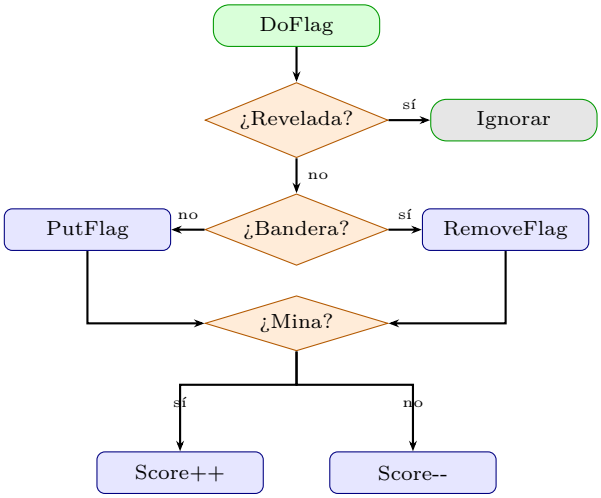


Figura 5. Lógica de colocación/remoción de banderas y puntuación.

La victoria se verifica en `CheckVictory`: si `Score == MineTotal`, se activa `VictoryF = 1` y el juego termina con el mensaje “GANASTE”.

V. Sistema de Colores RGB565

El formato RGB565 codifica cada color en 16 bits: 5 bits para rojo, 6 para verde y 5 para azul. La Tabla I muestra los colores utilizados.

Cuadro I
Paleta de colores RGB565 del juego.

Estado	HEX	RGB	Color
Oculta	`39E7`	(68, 68, 68)	Gris oscuro
Vacía (0)	`D6D6`	(211, 211, 211)	Gris claro
1 vecino	`001F`	(0, 0, 255)	Azul
2 vecinos	`0300`	(0, 128, 0)	Verde
3 vecinos	`7800`	(255, 0, 0)	Rojo
4 vecinos	`001F`	(0, 0, 139)	Azul oscuro
5 vecinos	`CA70`	(139, 0, 0)	Granate
6 vecinos	`0210`	(0, 255, 255)	Cian
7 vecinos	`D69A`	(238, 130, 238)	Violeta
8 vecinos	`FFFF`	(255, 255, 255)	Blanco
Mina	`0000`	(0, 0, 0)	Negro
Bandera	`FFE0`	(255, 255, 0)	Amarillo

Los colores son generados en Python y almacenados en `0x438`, evitando cualquier cálculo en MARIE. Al revelar una celda, `RevealCell` simplemente copia el valor precomputado al display.

VI. Patrones de Diseño Aplicados

Aunque el proyecto se desarrolla en bajo nivel, se identifican varios patrones de diseño clásicos adaptados al entorno MARIE.

VI-A. *Separación de Responsabilidades*

Python se encarga de toda la generación y precomputación de datos; MARIE se limita a la ejecución del juego. Esto reduce drásticamente la complejidad del ensamblador.

VI-B. *Modelo–Vista*

- **Modelo**: segmentos `0x238` (tablero real) y `0x538` (estado visible).
- **Vista**: segmento `0xF00` (display de video).

La actualización de la vista siempre pasa por `RevealCell`, que actúa como controlador.

VI-C. *Precomputación (Flyweight)*

La tabla de vecinos (`0x638`) y la tabla de colores (`0x438`) son calculadas íntegramente en Python. En MARIE, estas operaciones serían prohibitivamente costosas (requieren multiplicaciones y condicionales anidadas).

VI-D. *Stack Manual (Simulación de Recursividad)*

MARIE no posee pila de llamadas ni instrucción de recursión. El algoritmo DFS se implementa con un arreglo en `0xE00` y el puntero `StackTop`, simulando una pila LIFO con operaciones `STOREI`/`LOADI`.

VI-E. *Subrutinas con JNS*

La instrucción `JNS` (Jump and Store) guarda la dirección de retorno en la primera palabra de la subrutina y salta a la segunda. Esto permite modularizar el código en unidades reutilizables (`MulRoutine`, `RevealCell`, `PrintString`, etc.).

VII. Implementación y Resultados

VII-A. *Tablero Generado*

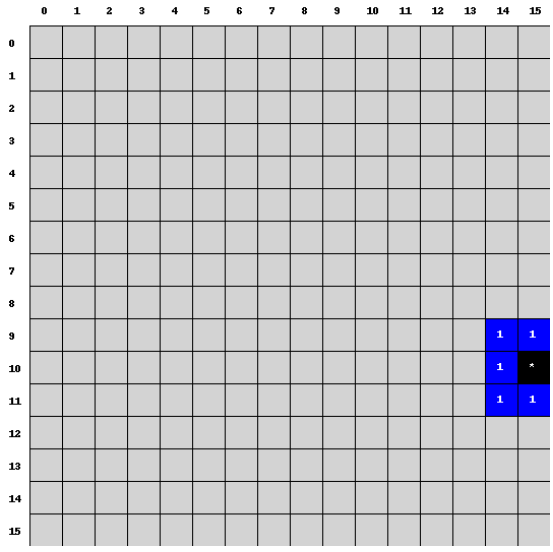


Figura 6. tablero_real.png generado por Python mostrando el tablero 16x16 con la mina en posición (10,15) y los valores 1 alrededor.

El tablero de prueba contiene 1 mina ubicada en la celda $(10, 15)$ (índice lineal 175). Las celdas adyacentes $(9, 14)$, $(9, 15)$, $(10, 14)$, $(11, 14)$ y $(11, 15)$ tienen valor 1. El resto del tablero es 0, lo que activa la expansión DFS al revelar cualquier celda vacía.

VII-B. *Ejecución en MARIE.js*

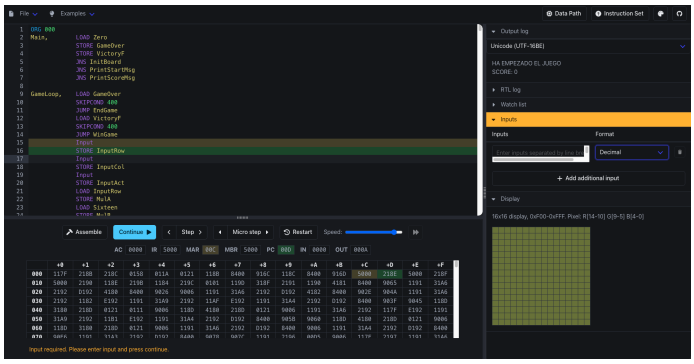


Figura 7. Captura del simulador MARIE.js con juego.mas cargado, mostrando el display 16x16 en estado inicial.

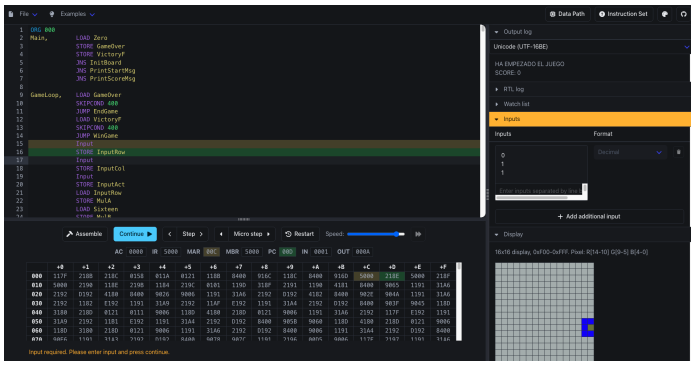


Figura 8. Captura del simulador MARIE.js mostrando el resultado después de revelar una celda vacía: la expansión DFS ha revelado la mayor parte del tablero en gris claro, con las celdas "1.en azul alrededor de la mina.

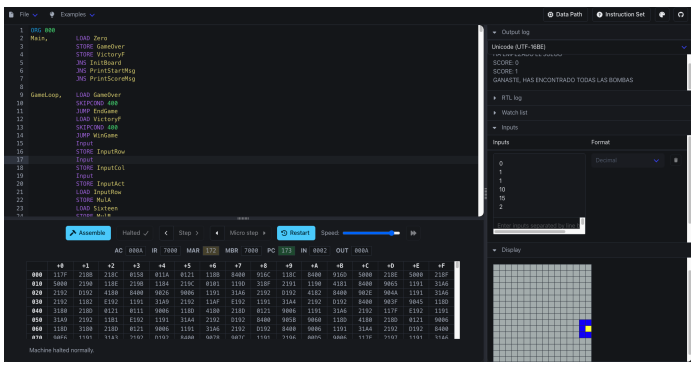


Figura 9. Captura del simulador MARIE.js mostrando el estado de victoria.

VII-C. Métricas del Programa

Cuadro II
MÉTRICAS DEL ARCHIVO `JUEGO.MAS`.

Métrica	Valor
Instrucciones de lógica (BombGame)	≈ 280
Palabras de datos (tablero 16×16)	256
Palabras de datos (minas)	256
Palabras de datos (colores)	256
Palabras de datos (estado visible)	256
Palabras de datos (vecinos)	2048
Tamaño total del programa	> 3500 palabras
Subrutinas implementadas	12

VII-D. Limitaciones Encontradas

- **Profundidad del stack DFS**: el stack en `0xE00` tiene capacidad limitada por el espacio hasta `0xF00` (256 entradas). Para tableros con muchas celdas vacías contiguas, esto podría desbordarse.
- **Números de dos dígitos**: `PrintSigned` soporta hasta dos dígitos decimales (suficiente para el rango de puntuación esperado).
- **Velocidad de multiplicación**: `MulRoutine` realiza hasta 255 iteraciones para calcular 15×16, lo que introduce latencia perceptible en el simulador.

VIII. Conclusión

Este proyecto demuestra que es posible implementar aplicaciones no triviales en arquitecturas simples como MARIE mediante una adecuada organización de memoria, precomputación de datos y algoritmos adaptados al conjunto de instrucciones disponible.

La separación entre Python (generación) y MARIE (ejecución) resultó fundamental: trasladar la complejidad computacional fuera del entorno limitado permitió implementar un juego completo con expansión DFS, sistema de banderas, puntuación y detección de victoria/derrota.

Se evidencia además que conceptos modernos como patrones de diseño (Modelo–Vista, Separación de Responsabilidades, Flyweight) son aplicables incluso en programación de bajo nivel, mejorando la claridad y mantenibilidad del código ensamblador.

Este trabajo refuerza la comprensión de cómo funcionan internamente los sistemas computacionales y cómo se pueden construir soluciones completas a partir de componentes básicos.

Cabe destacar que parte de la asistencia en el desarrollo del código ensamblador y la lógica del juego se realizó mediante modelos de lenguaje entrenados sobre el entorno MARIE.js, utilizando como modelos base **Claude Sonnet 4.5** (Anthropic) [5] y **MiniMax M2.7** (MiniMax AI) [6]. Esta integración permitió acelerar el proceso de diseño y depuración del programa en arquitectura MARIE.

Referencias

[1] Buscaminas Online. *Buscaminas – Juego online gratuito*. [Online]. Available: https://buscaminas.eu/ [Accessed: 2025].

[2] MARIE-js Contributors. *MARIE.js – A MARIE Simulator built in JavaScript*. Monash University, 2018. [Online]. Available: https://marie.js.org/ [Accessed: 2025].

[3] MARIE-js Contributors. *MARIE Instruction Set (with Opcodes) – MARIE.js Wiki*. GitHub, 2018. [Online]. Available: https://github.com/MARIE-js/MARIE.js/wiki/MARIE-Instruction-Set-(with-Opcodes) [Accessed: 2025].

[4] L. Null and J. Lobur, *The Essentials of Computer Organization and Architecture*, 4th ed. Jones & Bartlett Learning, 2014.

[5] Anthropic. *Claude Sonnet 4.5 – Large Language Model*. Anthropic, 2025. [Online]. Available: https://www.anthropic.com [Accessed: 2025].

[6] MiniMax AI. *MiniMax M2.7 – Large Language Model*. MiniMax, 2025. [Online]. Available: https://www.minimaxi.com [Accessed: 2025].